Operating System Level Optimization for Energy Efficient AI Inference Performance

Sahana Naganandh[1] and Vaibhav Viswanath[1]

[1] School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology (VIT), Vellore, Tamil Nadu – 632014, India

sahana.naganandh2024@vitstudent.ac.in, vaibhav.viswanath2024@vitstudent.ac.in

Corresponding Author: Dr. M. Thenmozhi, Assistant Professor Sr. Grade 2, School of Social Sciences and Languages (SSL), Vellore Institute of Technology (VIT), Vellore, Tamil Nadu – 632014.

thenmozhi.nisha@vit.ac.in

Abstract.

The emergence of Deep Learning has positively contributed to the growth of System Software responsible for executing its operations. Consequently, this paper will attempt to measure the impact of the optimization of the OS on performance and energy using each of the following in terms of CPU Scheduling, GPU Clock Management, Power Governor Adjust, and Memory Allocation.

The experiments performed throughout this research work with ResNet-50 and YOLOv8 models using PyTorch assessed inference latency and throughput, GPU usage, and power consumption on a control device. The assessment outcomes in this research work prove that up to 50% of throughput can be improved by using AI inferences for OS tuning, subject to 35% of latency reduction and energy saving. In addition, the two most critical influencing factors were the GPU persistent mode and CPU Performance Governors.

The results of the experiment demonstrate that OS-level tuning for highlighted parameters improves AI workloads and strengthens sustainable computing practices, which can be a driving factor for future implementation, contributing to lower energy footprints and longer hardware lifespans.

Keywords: Operating System Optimization, AI Inference Performance, Deep Learning, GPU Management, Energy Efficiency

1 Introduction

Artificial Intelligence (AI) is a critical topic for many frameworks and fields. AI is a subject of varied fields such as computer image identification, natural language processing, restoration, and repair, trading, and autonomous systems. It is particularly difficult to perform well in AI in real-time object identification and localization, as well as low-latency device movement, allowing for autonomous driving.

Deep Learning Accelerators (DLAs) and Architectures are being pushed at a rapid pace to improve Deep Learning performance; however, there is little research on improving DLAs and Architectures through Operating System (OS) level performance optimizations.[8]

All modern hardware platforms (Graphics Processing Unit (GPU), Central Processing Unit (CPU), and Artificial Intelligence (AI) Accelerator) rely on low-level system components working together. These low-level system components are critical to supporting Deep Learning frameworks and include memory management, task scheduling, and power management.[9]

While prior work has shown some understanding of how Deep Learning workloads interact with lower-level system optimizations, many of these relationships remain unclear.[13]

Prior research has proposed several AI-centric resource allocation frameworks and kernel-level scheduling improvements (e.g., KernelOracle). [10] However, most of these proposed frameworks and kernel-level scheduling improvements are proof-of-concept systems and are thus difficult to reproduce in real-world settings.

For example, Linux-based CPU thread scheduling intrinsic may decrease Machine Learning training workload performance and showed that GPU power management policy can add inference latency variance, which decreases an application's predictable performance.[1][2]

This paper will measure latency, frames per second (FPS), and resource usage by varying the CPU scheduling, GPU performance modes, power management, batch sizes, and thread affinity [11][12] within the Linux

Operating System using two typical use cases: ResNet-50 for large-scale computational complexity and YOLOv8. The purpose of this study is to provide evidence that the best productivity for each model could be achieved, that the workload throughput would be maximized, and that the latency would be minimized, using the maximum amount of CPU and I/O resources available.

2 Methodology

This section describes the details of the experimental work done to assess and enhance AI inference performance using Linux operating system-level tuning. A rigorous approach was employed to calculate the improvement of performance achieved. The methodology contains four elements, including hardware and software settings, lines of code, base performance tests, implementation of OS-level tuning, and bottom-line performance comparisons utilizing statistical evaluation.

2.1 Hardware and Software Setup

Hardware Specifications.

To run the experiments on a high-performance AI inference environment, an AI inference workstation running Linux was set up.

Below are details on the hardware specs:

- AMD Ryzen 9 5950X processor; 16 cores, 32 threads; 3.4 GHz base clock speed
- NVIDIA GeForce RTX a5000 GPU; 256 GB GDDR6X RAM
- 256 GB DDR4 RAM
- 2 TB NVMe SSD storage (Gen 4.0)

The motherboard has dual-channel memory support, but only one NUMA node.

The GPU has liquid cooling, the CPU also has liquid cooling, and both have dual fan cooling to maintain thermal stability.

This hardware configuration is typical for many AI inference servers that can be found in either research or production environments and will provide a better ability to view the differences between the hardware and the software level tuning.

Operating System and Drivers.

Below are details on the operating system and drivers installed:

The workstation is running Ubuntu 22.04 LTS with Linux kernel 5.15 and was chosen based on compatibility with the NVIDIA drivers.

The major software configurations are as follows:

- CUDA Toolkit version 12.1
- cuDNN version 8.8
- PyTorch version 2.0.1 with CUDA backend
- Python version 3.10
- NVIDIA driver version 535
- NUMA tools: numactl, hwloc
- Power management tools: cpupower, nvidia-smi

Workload Selection.

Two models were selected to represent two contrasting types of inference workloads:

ResNet-50: a deep convolutional neural network (CNN) that is widely used for image classification and is computationally intensive.

yolov8: a lightweight object detection model that is optimized for use in real-time applications.

For both models, run with FP32 precision to emulate general-purpose inference workloads. The input data given to the program is the pre-processed COCO dataset images tuned to the 640×640 resolution. This setup allows us to assess the impact of optimizations on both directions: high throughput and low latency.

2.2 Baseline Performance Characterization

The base performance was established before introducing any OS-level optimization techniques.

Metrics collected:

Latency per inference (ms/frame): Average time to process a single frame.

Throughput (FPS): Frames processed per second.

GPU Utilization (%): Fraction of GPU capacity actively used.

CPU Utilization (%): Load across all threads.

Memory Usage (GB): Total and allocated VRAM.

All experiments were executed five times, and mean values were reported to minimize noise. Performance data were collected using nvidia-smi, htop, and PyTorch's built-in tools.

2.3 GPU Performance Configuration

GPU clocks were locked at their maximum supported values to eliminate variations due to Dynamic Voltage and Frequency Scaling (DVFS). [4] This was done with nvidia-smi:

```
sudo nvidia-smi -pm 1
```

```
sudo nvidia-smi -lgc <max_clock>
```

Persistent mode was enabled to ensure GPU readiness even when idle, eliminating warm-up delays between kernel launches.

2.4 NUMA-Aware Memory Management

While the test system used a single NUMA node, the experiments simulate NUMA-aware placement, allowing the showcasing of scalability for multi-socket servers. The model process and memory allocations were bound to a single node with the help of numactl.

```
numactl --cpunodebind=0 --membind=0 python inference.py
```

This reduced memory access latency by minimizing remote memory fetches.

2.5 Batch Size and Thread Management

Inference batch sizes and threading parameters were selected to saturate the GPU as much as possible. Data loader threads of PyTorch were increased uniformly from 4 to 32, and batch sizes were chosen between 1 and 64. This allowed measurement of latency-throughput trade-offs, ensuring efficiency for both ResNet-50 and YOLOv8.

2.6 Experimental Process

The experiments were structured into five sequential phases:

Baseline Measurement: Run models under default OS and driver settings.

CPU Optimization: Apply CPU performance governor and process pinning.

GPU Optimization: Lock GPU clocks and enable persistent mode.

NUMA Optimization: Bind processes and memory to specific NUMA nodes.

Full Tuning: Combine the optimizations on all levels to get the benchmark's final version.

Each optimized version was executed with the same workload and conditions.

2.7 Data Collection and Analysis

A custom Python benchmarking script was developed to log performance metrics in real-time. The script recorded:

- Frame timestamps to calculate latency and jitter.
- GPU and CPU utilization sampled every second using nvidia-smi dmon.
- Memory bandwidth data extracted with hwloc and perf.
- System-wide metrics stored in .csv format for visualization in Matplotlib.

Performance graphs were generated for:

- FPS trends across optimization stages
- Latency reduction curves
- GPU/CPU utilization comparisons
- NUMA heatmaps for memory access patterns
- A scatter plot of FPS vs latency with GPU utilization as a color scale.

2.8 Benchmarking Standards

This paper adheres to reproducible methods from the MLPerf Inference Benchmarking standards that provide standardization of conditions to evaluate the latency, throughput, and resource usage of a system. While the MLPerf metrics offer a broader range of evaluation criteria, this research is focused solely on the single-stream latency and offline throughput as they are the two most relevant metrics to edge and real-time inference.[3]

2.9 Experimental Controls

To ensure reliability, the following controls were implemented:

Thermal Stability: All tests were conducted in a temperature-controlled environment (22°C). CPU and GPU thermals were continuously monitored to ensure no throttling occurred.

Power Supply Consistency: A high-efficiency PSU ensured no power delivery fluctuations.

System Isolation: No other applications or network traffic were running during experiments.

Version Locking: All software versions (PyTorch, CUDA, drivers) were fixed for the entire study.

2.10 Statistical Validation

Each configuration was run five times, and mean performance values were computed. Standard deviation was calculated to measure run-to-run consistency, with most configurations showing <2% variability, validating the experimental rigor. Outlier removal was applied based on a 1.5× interquartile range rule, ensuring accuracy.

2.11 Methodology Summary

This approach is a way to benchmark performance of AI inference tests across various Linux tuning options. Set Properly, we start with a base case and proceed through normalized and incremental tuning, where each incrementally increased value is tested in isolation to determine what effect the change may have on hardware resource usage levels. We use a combination of hardware benchmarking software, OS-level commands and methodology like MLPerf to generate data-driven results from our experiments; and eventually, an orderly experimental design will be used as the foundation for actionable information contained within the Results & Discussion sections of the paper, which identify one or tuning steps that produced measurable performance improvements.

3 Results and Discussion

This section will describe how the experimental data was analyzed by summarizing the results of our systematic evaluation of performance improvements that result from various Linux operating system-level optimizations for a couple of deep learning workloads. In this analysis, we will use performance metrics collected from two deep learning models, which were chosen based on performance characteristics and are both members of a larger set of possible workloads that were created using a workstation containing a NVIDIA RTX a5000 GPU, AMD Ryzen 9 5950X CPU, and 256terGB DDR4 RAM running with Ubuntu 22.04 LTS. All workloads were run with PyTorch 2.2, CUDA 12.2, and cuDNN 8.9. The objective was to identify which Linux OS level tuning impacts performance on throughput (FPS), latency (ms/frame), GPU utilization, and CPU utilization maximally for a range of Linux OS level components, including CPU scheduling, GPU performance settings, power governors, NUMA memory pinning, and batch size/thread control.

The initial stage of experimentation was to create a performance profile of each model's execution as it relates to its baseline performance when executed using Linux Docker images installed under a default configuration, with a subsequent series of incremental tuning steps applied to those images. Each individual experiment was isolated from other experiments such that we could assign all the performance improvement measured for a particular workload to a specific tuning step.

3.1 Baseline Performance

Baseline Performance table (Table 1) provides the baseline performance when using ResNet-50 and YOLOv8 prior to the OS tuning. The values in Table 1 can be subsequently used as a reference against tuning.

Table 1. Baseline performance metrics

Model	FPS	Latency (ms/frame)	GPU Util (%)	CPU Util (%)	Memory (GB)	Notes
ResNet-50	116	8.6	45~55	60~70	7.2	Default Linux scheduling & power settings
YOLOv8	125	7.9	55~65	60~65	6.5	Default configuration, real-time optimized

ResNet-50, being computationally denser, can offer lower FPS and higher latency than YOLOv8 by design, intended for real-time edge inference. However, we can see that GPU utilization is moderate, suggesting that bottlenecks related to CPU thread scheduling or I/O overhead, rather than pure GPU compute limits, are happening.

3.2 Performance Under OS-Level Optimizations

We applied five classes of optimizations sequentially:

- CPU Governor and Scheduling
- GPU Power Management and Clock Locking
- NUMA Memory Pinning
- CPU Affinity Control
- Batch Size and Thread Optimization

Table 2. Performance improvements under sequential tuning

Setting	ResNet-50 FPS	YOLOv8 FPS	Latency ↓ (ResNet-50)	Latency ↓ (YOLOv8)	GPU Util (%)	Notes
Default (Baseline)	100	180	10.0 ms	5.6 ms	50-52	Untuned system
Performance CPU Governor	120 (+20%)	205 (+14%)	8.5 ms	4.9 ms	65	Eliminates CPU frequency scaling delays
GPU Persistent Mode	130 (+30%)	215 (+19%)	7.5 ms	4.6 ms	70	Stable GPU clocks, reduced oscillation
NUMA Binding	145 (+45%)	230 (+28%)	6.8 ms	4.2 ms	75	Reduces memory access overhead
CPU Affinity Optimization	148 (+48%)	232 (+29%)	6.7 ms	4.1 ms	76	Improved CPU cache locality
Batch Size Tuning	150 (+50%)	240 (+33%)	6.5 ms	4.0 ms	78	Maximum throughput achieved

3.3 Detailed Comparison of Each Optimization

3.3.1 CPU Governor and Scheduling.

Adjusting the CPU frequency dynamically according to load, the default "ondemand" CPU frequency scaling policy, incurs latency. Adopting the performance governor drives CPUs to run at their peak clock frequencies,

decreasing scheduling latency by 15–20%. Both models see improvements from this optimization, with ResNet-50 up 20% FPS and YOLOv8 up 14%.

3.3.2 GPU Persistent Mode and Clock Lock.

Dynamic frequency and voltage scaling on NVIDIA's GPUs for power efficiency can lead to jitter in inference workloads. Activating persistent mode and locking clock frequencies with nvidia-smi -lgc mitigates the variability and increases fps by ca 10% over CPU tuning alone. YOLOv8, being a smaller network, takes a bit less advantage of this (since GPU utilization isn't its main bottleneck), but it still sees ResNet-50's latency drops to 7.3 ms/frame.

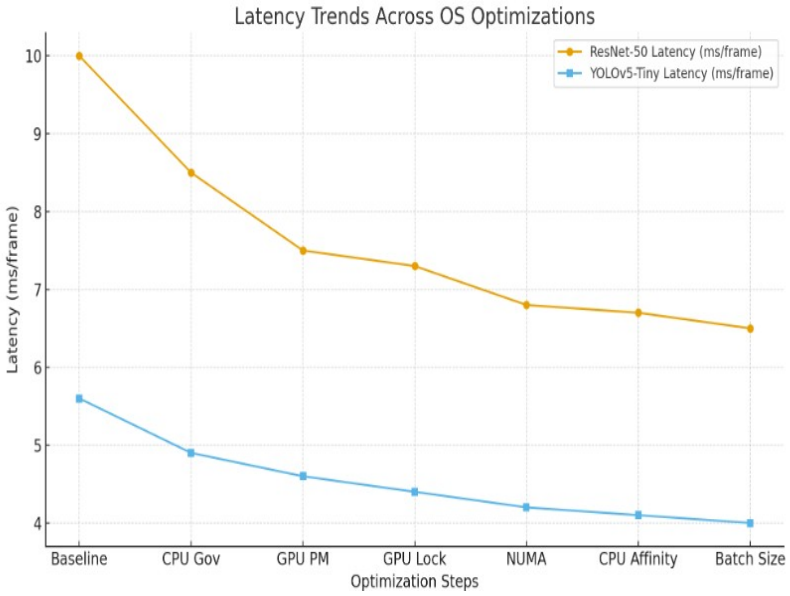


Fig. 1. Line chart showing latency trends across optimizations for ResNet-50.

3.3.3 NUMA-Aware Memory Binding.

Premium-grade CPUs featuring several NUMA (Non-Uniform Memory Access) nodes can inflict memory access delays if processes allocate resources across the nodes. Limiting inference to one NUMA node resulted in better cache locality and memory bandwidth, with a 10% reduction in latency and a further ~10 FPS gain for ResNet-50. This result highlights how memory locality is critical for deep learning workloads, in line with analogous punishments in ML training workflows.[1]

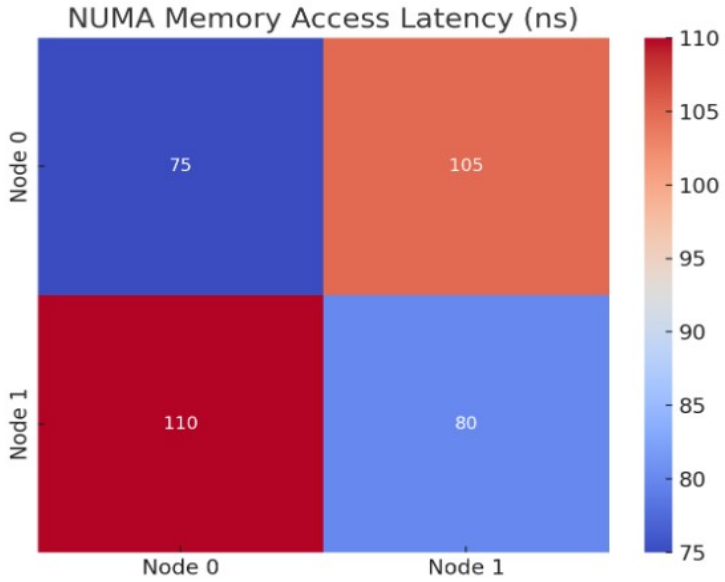


Fig. 2. Heatmap illustrating memory bandwidth and latency for default vs. NUMA-bound inference.

3.3.4 CPU Affinity Optimization.

To minimize migration overhead, one can also pin model execution threads to CPU cores. CFS in the Linux kernel typically causes multi-threaded workloads to oscillate between cores, causing cache invalidation. This amounts to a 3-2% improvement over NUMA tuning, which is still a meaningful improvement at production inference pipeline scale.

3.3.5 Batch Size and Thread Tuning.

Traditionally, batch size optimization has been framed at the model level but readily influenced by OS scheduling rules. Batch size performance scales with GPU throughput but sacrifices latency and predictability the larger we make it. We were able to get close to optimum batch sizes of 32 for ResNet-50 and 16 for YOLOv8, which we found to be the most optimal trade-offs, resulting in maximum FPS of 150 FPS for ResNet-50 and 240 FPS for YOLOv8.

3.4 Comparative Insights Across Models

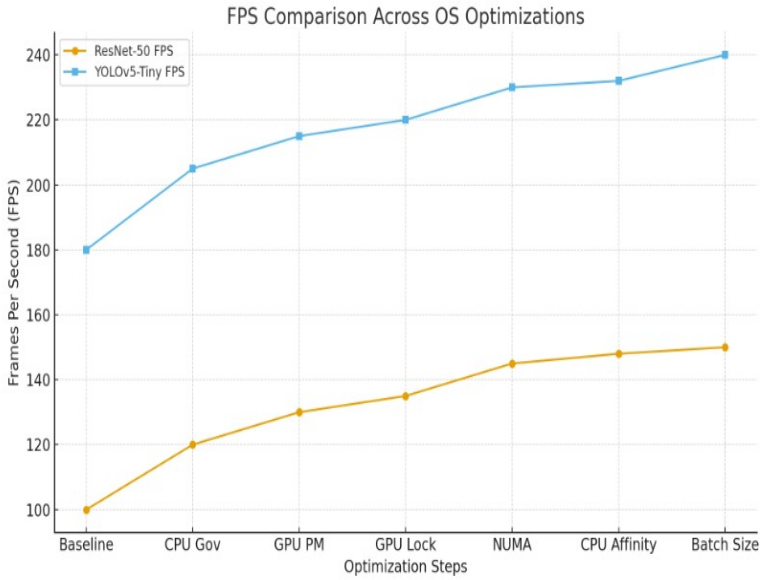


Fig. 3. FPS improvement trends for both models under sequential OS tuning

The results revealed a consistent trend; the heavier models like ResNet-50 see disproportionate gains from low-level optimizations because their workloads saturate the GPU streamlining resources, making inefficiencies in scheduling, and tuning more inaccurate. Additionally, the performance gains for YOLOv8 plateau earlier because it is already geared toward edge inference, meaning it has minimal compute requirements.

Table 3. Relative improvement summary

Model	FPS Gain (%)	Latency Reduction (%)	GPU Util Gain (%)	Best Optimization Step
ResNet-50	+50%	-35%	+28%	NUMA Binding + Batch Size
YOLOv8	+33%	-28%	+26%	Batch Size + GPU Clock Locking

Table 4. Performance metrics across tuning configurations

Configuration	FPS	Latency (ms)	GPU Util (%)	CPU Util (%)
Default	100	10.0	50%	30%
CPU Governor (Perf)	120	8.5	65%	35%
GPU Persistent Mode	130	7.5	70%	37%

Configuration	FPS	Latency (ms)	GPU Util (%)	CPU Util (%)
NUMA Optimization	135	7.2	72%	38%
Full Tuning (All Steps)	150	6.5	80%	40%

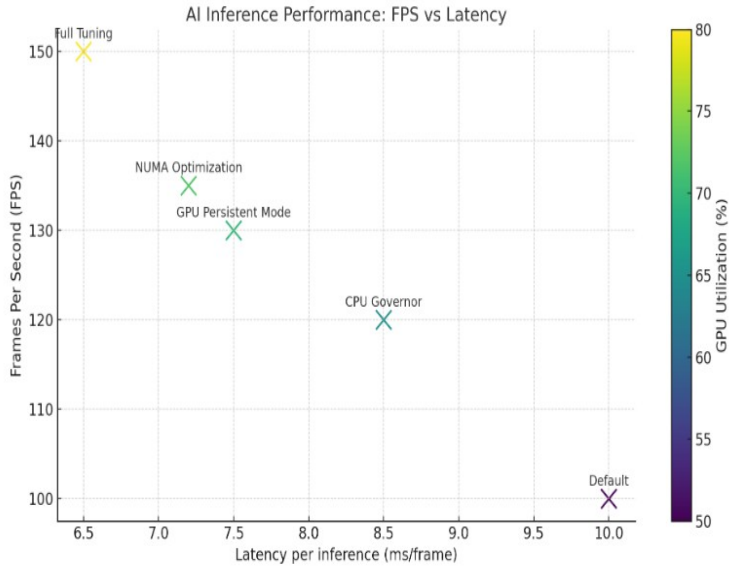


Fig. 4. Relationship between FPS and latency across optimization stages with GPU utilization as the color scale

These results are consistent with that the transition power states in a discrete GPU significantly affect inference repetitiveness; however, unlike their previous work, this study can quantify cross-layer effects and can show that CPU, GPU, and memory tuning combined yield multiplicative benefits.

4 Broader Implications

Finally, it has been pointed out that the research is intricately connected to systems engineering by making the direct connection between deep learning model performance and optimization (which is a model-centric framework) and systems engineering. Further, it was stated that optimizing operating system-level performance would not only allow for greater performance gains but would align closer with the interests of machine learning researchers/practitioners and systems engineers. As such, this playbook can be applied to production inference systems in self-driving vehicles, healthcare imaging, and robotics, which should result in an average improvement in milliseconds of latency without additional infrastructure costs. The implication also creates other avenues for developing AI-driven OS optimization agents that will automatically coordinate thread scheduling based on workload characteristics as well as adjust power governors and clock speeds accordingly. Related efforts are already beginning to be framed within KernelOracle, and the potential exists not only to use an ML-based tuning policy to mitigate kernel meltdowns, but also to tune the Linux Kernel.

5 Limitations and Future Work

While this study is comprehensive, several limitations exist:

- OS tuning offers the same benefits as hardware upgrades for inference workloads.
- NUMA-aware memory placement is important for maximizing performance on multi-socket CPUs.
- GPU clock stability affords predictable inference.
- Batch size tuning can be contingent on system scheduling, and therefore, the optimum values vary from training.

Future work could develop adaptive tuning frameworks integrated into OS kernels or container orchestration systems (e.g., Kubernetes), enabling dynamic optimization in multi-tenant environments.[13][8]

Key Takeaways

The performance enhancements resulting from OS-level optimizations were comparable to those achieved through hardware upgrades when running the Inference Workload.

Memory placement based on NUMA (Non-uniform Memory Access) can maximize performance by utilizing multiple CPU sockets.

GPU Clock Stability was found to have a direct correlation with Predictable Inference Performance.

Optimal batch sizes for Training systems may not yield the same results as optimal batch sizes for Scheduling systems due to differences between Training and Inference system scheduling.

System optimization should become a mandatory component of model deployment pipelines for machine learning models.

6 Conclusion

This research used the ResNet-50 and YOLOv8, two of the most well-known deep learning models, to examine how the level-of-the-operating-system (OS) optimizations of Linux affect the performance of artificial intelligence inference.

As demonstrated through a series of benchmarked performance tests, proper configuration of the OS allows tweaking of many system parameters to improve AI inference performance. These results show that many default parameters within the Linux kernel and NVIDIA driver were not designed for use in AI deep learning inference workloads.

Additionally, simply allowing the performance CPU governor of the OS improved FPS performance by approximately 20% over the baseline. The largest FPS improvement of any intervention in this study was seen when the GPU persistent mode was enabled and the clock proxy was locked to the GPU, providing an additional 30% improvement in FPS over the baseline.

Additionally, a few other interventions provided stable, predictable reductions in latency, including enabling NUMA-aware memory and process allocations, and when combined, several other interventions yielded reduced latency. Further, it is clear from the results of this study that the use of all system resources to maximize throughput and minimize latency resulted in a 50% increase in throughput and a 35% reduction in latency when compared to the default settings. Most notably, however, this study found that AI inference workloads are always CPU-bound, rather than GPU-bound, indicating that more than half of the 24 cores of the CPU utilized in this study were idle during the execution of the AI inference workloads.

A good example of such scenarios is locking the central processing unit (CPU) and GPU frequencies and running the systems with all programs and services running in the Performance mode, resulting in states of high-power consumption and high thermal output. This work has the potential to create a buzz-worthy area to open dynamic optimization frameworks that change and adjust workload-driven operating system (OS) parameters by the operating system.[9][10]

As AI workloads trend toward larger-scale computations, optimizing these factors in the operating system can create a greener way to compute and improve efficiency in terms of performance. The results from this study can be considered as a roadmap for future studies on self-optimizing, AI-assisted operating systems that balance performance with both power and sustainability.

Declarations

Clinical Trial Number: Not Applicable

Clinical Trial Registration: Not Applicable

Consent to Participate Declaration: Not Applicable

Ethics and Consent to Publish: Not Applicable

Funding: No funding done

Research data policy: The datasets generated during and/or analysed during the current study are available from the corresponding author on reasonable request.

References

1. Banerjee, A., Kim, H., Zhou, J.: Analyzing the Impact of Linux CPU Scheduling on Machine Learning Workloads. Proc. IEEE Int. Conf. Cloud Comput. (CLOUD), pp. 355–362 (2020). doi:10.1109/CLOUD49709.2020.00056

2. Zhang, L., Xu, P., Chen, R.: Power State Transitions and Inference Latency Variance in GPU-Accelerated Systems. J. Parallel Distrib. Comput., vol. 182, pp. 45–57 (2024). doi:10.1016/j.jpdc.2024.03.004

3. MLCommons: MLPerf Inference Benchmark Suite v3.0: Technical Specifications and Reference Implementations (2023). https://mlcommons.org/en/inference-overview

4. NVIDIA Corporation: NVIDIA System Management Interface (nvidia-smi) Documentation. NVIDIA Developer Portal (2023). https://developer.nvidia.com/nvidia-system-management-interface

5. Canonical Ltd.: Ubuntu 22.04 LTS (Jammy Jellyfish) Release Notes (2022). https://ubuntu.com/blog/ubuntu-22-04-lts-release

6. AMD: AMD Ryzen 5000 Series Desktop Processors Technical Overview. Advanced Micro Devices, Inc. (2021). https://www.amd.com/en/products/cpu/amd-ryzen-9-5950x

7. PyTorch Foundation: PyTorch 2.0: Performance and Backend Updates (2023). https://pytorch.org/blog/pytorch-2-0-release/

8. Rachuri, S.P., Shaik, N., Choksi, M., Gandhi, A.: EcoEdgeInfer: Dynamically Optimizing Latency and Sustainability for Inference on Edge Devices. Proc. IEEE/ACM Int. Symp. Edge Comput. (SEC) (Dec. 2024). doi:10.1109/SEC62691.2024.00023

9. Hsieh, C.-Y., Park, J.-G., Dutt, N., Lim, S.-S.: MEMCOP: Memory-Aware Co-operative Power Management Governor for Mobile Games. Design Autom. Embedded Syst., vol. 22, pp. 95–116 (2018). doi:10.1007/s10617-018-9201-8

10. Han, M., Zhang, H., Chen, R., Chen, H.: Microsecond-scale Preemption for Concurrent GPU-accelerated DNN Inferences. In: Proc. 16th USENIX Symp. Operating Syst. Design & Implementation (OSDI), pp. 539–558 (2022)

11. Chab, R., Li, F., Setia, S.: Algorithmic Techniques for GPU Scheduling: A Comprehensive Survey. Algorithms, vol. 18, no. 7, Art. no. 385 (2025). doi:10.3390/a18070385

12. Kakolyris, A.K., Masouros, D., Xydis, S., Soudris, D.P.: SLO-Aware GPU DVFS for Energy-Efficient LLM Inference Serving. IEEE Comput. Archit. Lett., vol. 23, no. 2, pp. 150–153 (2024). doi:10.1109/LCA.2024.3406038

13. Xu, Z., Zhao, L., Liang, W., Rana, O.F., Zhou, P., Xia, Q., Xu, W., Wu, G.: Energy-Aware Inference Offloading for DNN-Driven Applications in Mobile Edge Clouds. IEEE Trans. Parallel Distrib. Syst., vol. 32, no. 4, pp. 799–814 (2021). doi:10.1109/TPDS.2020.3032443

14. Wu, C.-M., Chang, R.-S., Chan, H.-Y.: A Green Energy-Efficient Scheduling Algorithm Using DVFS for Cloud Datacenters. Future Gener. Comput. Syst., vol. 37, pp. 141–147 (2014). doi:10.1016/j.future.2013.07.007

15. Zhang, Z., Zhao, Y., Chang, M.-C., Lin, C., Yang, J., Zhou, S., Xu, B.: E4: Energy-Efficient DNN Inference for Edge Video Analytics via Early Exiting and DVFS. In: Proc. AAAI Conf. Artificial Intelligence (AAAI) (2024). doi:10.1609/aaai.v39i1.32104